



Département mathématiques et informatique
UFR des Sciences

TP9 “Représentation d’images à l’aide des quadrees”

Un **quadtree** est un arbre tel que tout nœud possède 0 ou 4 fils. Cette structure est très utilisée pour représenter des images sous une forme concise. Par la suite, on ne considère que des images carrées.

1 Traitement récursif des quadrees

On définit le type quadtree dans toute sa généralité par :

```
data Qtree a = Leaf a |  
             Node (Qtree a) (Qtree a) (Qtree a) (Qtree a)  
             deriving (Show, Ord, Eq)
```

Pour vous familiariser avec cette notion, vous pouvez utiliser la fonction `showQtree` définie dans le fichier `initQuadTree.hs` fourni sur la page ecampus pour ce TP.

```
*Main> putStr (showQtree (Leaf 1))  
Leaf 1  
*Main> putStr (showQtree q1)  
Node  
  Node Leaf 11 Leaf 12 Leaf 13 Leaf 14  
  Leaf 2  
  Leaf 3  
  Leaf 4  
*Main> putStr (showQtree q2)  
Node  
  Leaf 1  
  Leaf 2  
  Node  
    Leaf 31  
    Leaf 32  
    Node Leaf 331 Leaf 332 Leaf 333 Leaf 334  
    Leaf 34  
  Leaf 4
```

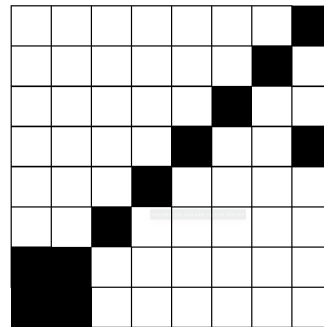
1. Définir les fonctions **nbLeaf**, **nbNoeuds** et **profondeur** qui déterminent respectivement : le nombre de feuilles, le nombre de nœuds (internes) et la profondeur d’un quadtree.

2 Représentation d'une image à l'aide d'un quadtree

Une image (carrée) de taille k possède $2^k \times 2^k$ pixels et peut être représentée par un couple (taille (= k), quadtree) :

- si la couleur de l'image est uniformément noire (resp. blanche), alors le quadtree est réduit à une feuille indiquant la couleur (**Leaf N**) (resp. (**Leaf B**)).
- sinon, on partage l'image de taille k en 4 images de taille $(k - 1)$ qui seront désignées par nord-ouest : **nw**, nord-est : **ne**, sud-ouest : **sw** et sud-est : **se**. Le codage associé est donc : (**Node nw ne sw se**). Chaque partie sera elle-même représentée par un quadtree. Si l'une de ces parties est uniformément blanche ou noire, alors le quadtree associé est une feuille contenant la couleur sinon cette partie est elle-même divisée en 4 sous-parties, et ainsi de suite récursivement...

Exemple : on considère l'image suivante :



- Cette image de taille $k=3$ est partagée en 4 images de taille 2. Tous les pixels de **nw** sont blancs (idem pour **se**).
- Le quadtree associé est donc de la forme (**Node (Leaf B) ne sw (Leaf B)**).
- Le carré **sw** se décompose à son tour en : (**Node (Leaf B) ne' (Leaf N) (Leaf B)**).
- **ne'** se décompose à son tour en : (**Node (Leaf B) (Leaf N) (Leaf N) (Leaf B)**).
- **sw** se décompose donc en :

```
(Node (Leaf B)
      (Node (Leaf B) (Leaf N) (Leaf N) (Leaf B))
      (Leaf N)
      (Leaf B))
```

- Enfin, **ne se** décompose en :

```
(Node (Leaf B)
      (Node (Leaf B) (Leaf N) (Leaf N) (Leaf B))
      (Node (Leaf B) (Leaf N) (Leaf N) (Leaf B))
      (Node (Leaf B) (Leaf B) (Leaf B) (Leaf N)))
```

Questions :

1. Pourquoi un quadtree seul n'est-il pas suffisant pour coder une image ?
2. Pour une image de taille k , combien de pixels représente une feuille de profondeur i ?
3. Définir les types suivants dans votre code Haskell, on doit pouvoir afficher les valeurs de ces types dans un terminal :
 - (a) Un type **Pixel** qui prend deux valeurs : **B** et **N**.
 - (b) Un type **Image** qui est défini par un tuple comprenant la taille de l'image en fonction de k et un **quadtree** paramétré par le type **Pixel**.

4. Définir une fonction appelée `showPixel` qui prend en paramètre une valeur de type `Pixel` et retourne un caractère représentant cette valeur. Pour un pixel `B`, la fonction retourne le caractère `'*`, et pour un pixel `N`, la fonction retourne le caractère `'_'`.
5. Définir la fonction `transposer im` prenant en paramètre une image représentée à l'aide de quadtree et qui permet de transposer l'image.

```
*Main> transposer (1, Node (Leaf B) (Leaf N) (Leaf B) (Leaf B))
(1,Node (Leaf B) (Leaf B) (Leaf N) (Leaf B))
...
```

6. Définir la fonction (`reverseImage im`) qui permet de calculer le négatif d'une image `im` où chaque pixel blanc est transformé en un pixel noir et vice-versa.

```
*Main> reverseImage (1, Node (Leaf B) (Leaf N) (Leaf B) (Leaf B))
(1,Node (Leaf N) (Leaf B) (Leaf N) (Leaf N))
...
```

3 Représentation matricielle d'une image

On souhaite dans cet exercice transformer les images représentées par les quadrees en images représentées par une matrice carrée via une fonction `qtreeToMatrice :: Image -> (Matrice Pixel)` contenant `'*` quand le pixel est `B`, et `'_'` quand il est `N`.

1. Définir un type `Vecteur` paramétré avec une variable type `a` et défini par une liste de valeurs de type `a`. Puis, définir un type `Matrice` qui est une liste de valeurs de type `Vecteur`.
2. Définir une fonction `showMatrice` qui prend en paramètre une valeur de type `Matrice` et retourne la chaîne de caractères permettant d'afficher la matrice comme illustré dans l'exemple suivant.

```
-- La fonction \texttt{putStr} en Haskell prend une chaîne de caractères en
-- paramètre et l'imprime à l'écran (console) en ajoutant automatiquement
-- un saut de ligne pour les caractères '\n'.
Main> putStr (showMatrice [[B,N],[B,B]])
*_
**
```

3. Définir la fonction `vecteurDe` prenant deux paramètres, un entier `n` et une valeur `x`, et retournant une liste de longueur `n` contenant uniquement la valeur `x` répétée. Le type de la valeur `x` n'est pas spécifié.
4. En utilisant la fonction `vecteurDe`, définir une fonction `matriceDe` prenant deux paramètres, un entier `n` et une valeur `x`, et retournant une liste de listes formant une matrice $n \times n$ où chaque élément est égal à `x`.
5. Définir la fonction `qtreeToMatrice :: Image -> (Matrice Pixel)` qui transforme une image `(k,qt)` en une matrice carrée $2^k \times 2^k$ selon le quadtree `qt`. Vous pouvez définir plusieurs fonctions auxiliaires pour décomposer les transformations en utilisant la fonction `matriceDe`.

```
*Main> q3 = (2,(Node (Leaf B) (Node (Leaf B) (Leaf N) (Leaf B) (Leaf N)) (Leaf N) (Leaf B)))
*Main> qtreeToMatrice q3
```

```

[[B,B,B,N],[B,B,B,N],[N,N,B,B],[N,N,B,B]]
*Main> putStr (showMatrice (qtreeToMatrice q3))
***_
***_
--**
--**

*Main> putStr (showMatrice (qtreeToMatrice (2, Leaf B)))
****
****
****
****

```

4 Représentation d'une image à l'aide d'un quadtree :

L'objectif est de définir la fonction `matriceToQtree :: (Matrice Pixel) -> Image` qui permet de passer de la représentation matrice carrée ($2^k, 2^k$) de pixels à la représentation (k, qt) où k est la taille de l'image et qt le **quadtree** associé.

Exemples. on reprend les exemples de la section 3, il faudrait avoir :

```

Prelude>matriceToQtree [[B,B,B,N],[B,B,B,N],[N,N,B,B],[N,N,B,B]]
(2, Node (Leaf B) (Node (Leaf B) (Leaf N) (Leaf B) (Leaf N)) (Leaf N) (Leaf B))

Prelude>matriceToQtree [[N,N,N,N],[N,N,N,N],[N,N,N,N],[N,N,N,N]]
(2, Leaf N)

```

La définition de la fonction de `matriceToQtree` est donnée par le code suivant :

```

matriceToQtree :: (Matrice Pixel) -> Image
matriceToQtree xss = (taille xss, regroupe (construit (pixelToLeaf xss)))

```

où

- `pixelToLeaf` transforme les pixels dans la matrice afin de retourner une matrice de feuilles pour préparer l'arbre **quadtree**.
- `construit` transforme la matrice de feuilles en rajoutant un nœud intermédiaire pour chaque pavé de 4 feuilles (i.e. pixels).
- `regroupe` permet de regrouper un nœud intermédiaire ayant les mêmes feuilles dans une seule feuille et récursivement construire l'arbre **quadtree**.
- `taille` retourne la valeur de k qui correspond à la taille de l'image.

1. Trouver la signature de chacune des fonctions auxiliaires de `matriceToQtree`.
2. Donner la définition de chaque fonction dans l'ordre suivant `pixelToLeaf`, `construit`, `regroupe` puis à la fin `taille`.